SYSDESIGN REFERENCE

The Thundering Herd Problem

A senior-level reference with tradeoffs, failure modes, and interview follow-ups for the thundering herd problem.

Prevent many clients from repeating the same expensive work when shared state expires or a dependency recovers.

VERSION 1.0 REVIEW
REVIEWED THROUGH 2026-07-18
STATUS HUMAN REVIEW REQUIRED

SYSDESIGN / THUNDERING-HERD

SYSDESIGN

The Thundering Herd Problem

CORE IDEA

Prevent many clients from repeating the same expensive work when shared state expires or a dependency recovers.

Reference architecture decision chain

01	02	03	04
Jitter	Single-Flight	Stale-While-Revalidate	Admission Control

Decisions to defend

+ Jitter
+ single-flight
+ stale-while-revalidate
+ admission control
+ and warmup.

Failure modes to test

+ Synchronized TTLs
+ retry alignment
+ and recovery overload.

Interview flow

- [ ] Clarify users, critical paths, and non-goals
- [ ] Estimate only scale that changes a decision
- [ ] Define API, data model, and source of truth
- [ ] Draw synchronous and asynchronous paths
- [ ] Pressure-test timeout, retry, overload, and recovery
- [ ] Close with tradeoffs and one alternative

Deep-dive drill

The Thundering Herd Problem

Probe	What a strong answer makes explicit
How does the design behave during a regional dependency failure?	Requirement, decision, cost, failure behavior, and observable metric.
Which metric reveals the first bottleneck?	Requirement, decision, cost, failure behavior, and observable metric.
What consistency does the user actually observe?	Requirement, decision, cost, failure behavior, and observable metric.
How would the design change at ten times the traffic?	Requirement, decision, cost, failure behavior, and observable metric.

Data questions

+ Partition key
+ Versioning
+ Uniqueness
+ Retention
+ Reconciliation

Operations questions

+ SLO
+ Backlog
+ Capacity
+ Alert
+ Failover test

DESIGN DISCIPLINE

For every box: which requirement justifies it, what cost does it introduce, and what happens when it fails?

PUBLISHING STATUS

REVIEW GATE: TECHNICAL REVIEW. AI-produced draft. A named human must verify technical accuracy, sourcing, freshness, and editorial quality before publication. Reviewed-through date: 2026-07-18.